PRAKTIKUM
SISTEM BASIS DATA

Nama	**Abyan Ammar Zaki**	**No. Modul**	**8**
NPM	**2406421005**	**Tipe**	**CS**

1.

Langkah	Screenshot

redis.js	completed_backend > src > database > JS redis.js > ...
	`const Redis = require('ioredis');`
	
	`const redis = new Redis({`
	`  host: '127.0.0.1',`
	`  port: 6379`
	`});`
	
	`module.exports = redis;`

Register	POST http://localhost:3000/user/register Send Docs Params Auth Headers (8) Body Scripts Settings Cookies raw JSON Schema Beautify { "name" : "Ammar", "username": "Ammar123", "email" : "ammar@gmail.com", "phone" : "+62 123-4567-8910", "password" : "Ammarg@m1nghehehaw" } Body 201 Created 318 ms 1.25 KB Save Response JSON Preview Visualize { "success": true, "message": "User registered successfully", "payload": { "id": 6, "name": "Ammar", "username": "Ammar123", "email": "ammar@gmail.com", "phone": "+62 123-4567-8910", "balance": 0, "created_at": "2026-04-16T02:05:53.174Z" } }

auth/login	POST http://localhost:3000/auth/login Send
	Docs Params Auth Headers (8) Body Scripts Settings Cookies
	raw JSON Schema Beautify
	{
	"email" : "ammar@gmail.com",
	"password" : "Ammarg@m1nghehehaw"
	}
	Body 200 OK 360 ms 1.39 KB Save Response
	JSON Preview Visualize
	{
	"success": true,
	"message": "Login successful",
	"payload": {
	"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjYsImVtYWlsIjoiYW1tYXJAZ21haWwuY29tIiwiaWF0IjoxNzc2MzA1MjM2LCJleHAiOjE3NzYzOTE2MzZ9.MDQ9AwioRT3v4qgP650PsuBJ63ieCr8fb_PTWm8sFEA",
	"user": {
	"id": 6,
	"name": "Ammar",
	"username": "Ammar123",
	"email": "ammar@gmail.com",
	"phone": "+62 123-4567-8910",
	"balance": 0
	}
	}
	}

Token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjYsImVtYWlsIjoiYW1tYXJAZ21haWwuY29tIiwiaWF0IjoxNzc2MzA1MjM2LCJleHAiOjE3NzYzOTE2MzZ9.MDQ9AwioRT3v4qgP650PsuBJ63ieCr8fb_PTWm8sFEA

2.

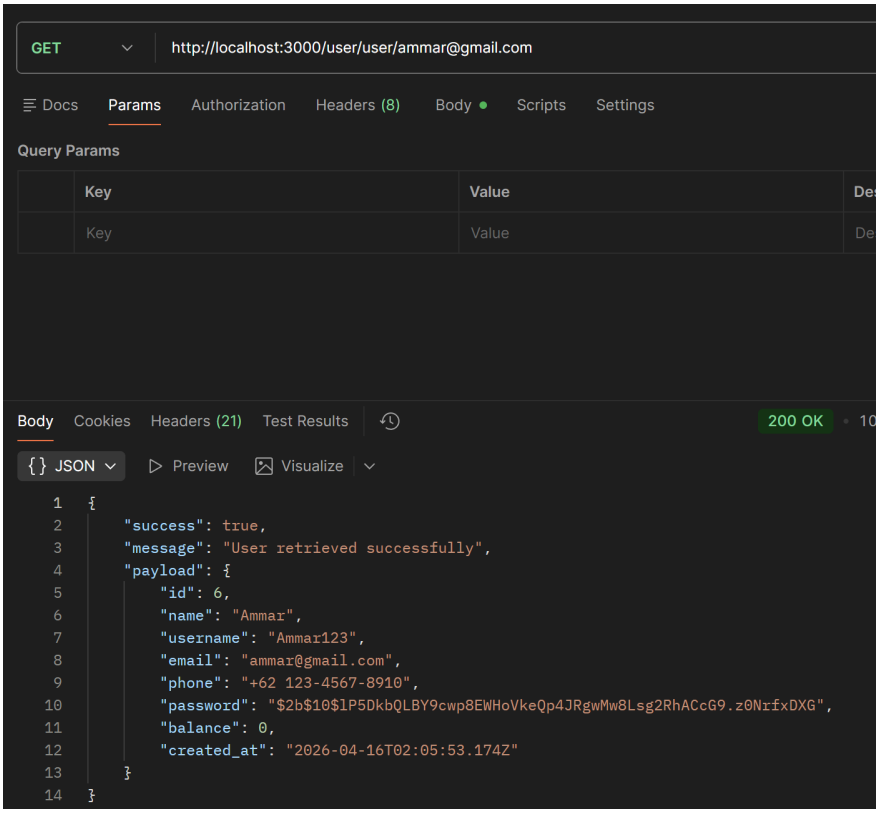
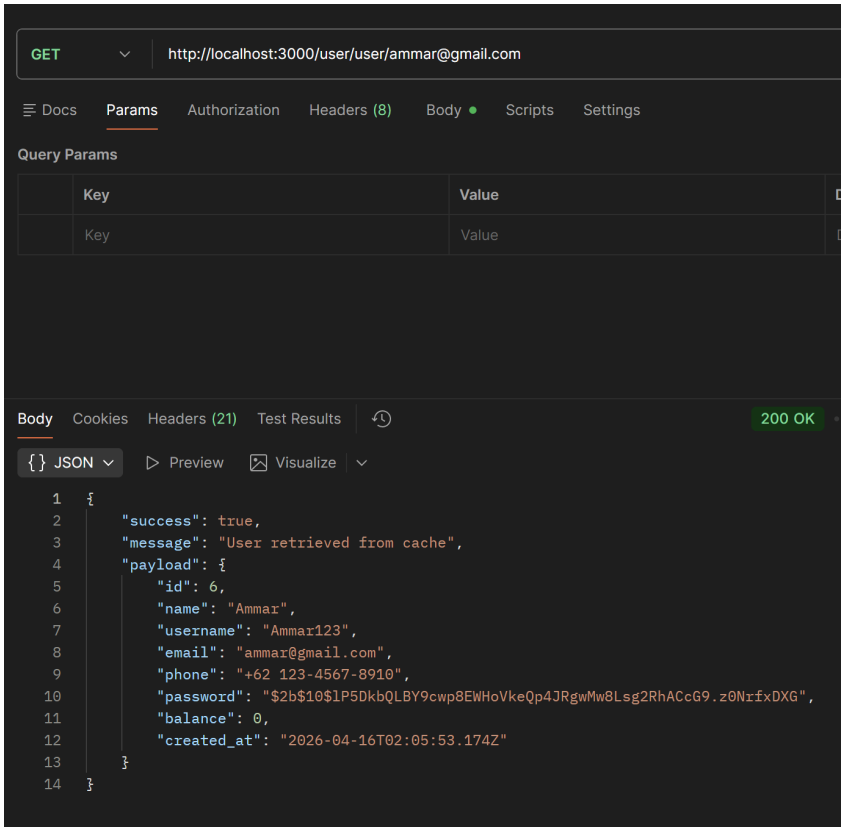
Langkah	Screenshot
user controller	
route	

```
static async getUserByEmail(req, res, next) {
  try {
    const { email } = req.params;
    const cacheKey = `user:${email}`;

    const cachedUser = await redis.get(cacheKey);

    if (cachedUser) {
      return res.status(200).json({
        success: true,
        message: 'User retrieved from cache',
        payload: JSON.parse(cachedUser),
      });
    }
    const user = await UserService.getUserByEmail(email);
    await redis.set(cacheKey, JSON.stringify(user), 'EX', 60);
    res.status(200).json({
      success: true,
      message: 'User retrieved successfully',
      payload: user,
    });
  } catch (error) {
    next(error);
  }
}
}
```

```
router.get('/user/:email', UserController.getUserByEmail);
```

Postman (percobaan pertama)	
Postman (percobaan kedua sebelum 60 detik)	

3.

Langkah	Screenshot

user controller	`static async updateProfile(req, res, next) {`
	`  try {`
	`  const { id, name, username, email, phone, password, balance } = req.body;`
	`  const updatedUser = await UserService.updateProfile(id, { name, username, email, phone, password, balance });`
	
	`  const cacheKey = `user:${email}`;`
	`  await redis.del(cacheKey);`
	
	`  res.status(200).json({`
	`  success: true,`
	`  message: 'User updated successfully',`
	`  payload: updatedUser,`
	`  });`
	`  } catch (error) {`
	`  next(error);`
	`  }`
	`}`

Postman update cache	PUT http://localhost:3000/user/update
	Docs Params Authorization Headers (9) Body Scripts Settings
	none form-data x-www-form-urlencoded raw binary GraphQL JSON
	`{ "id": 1, "name": "Ammar Updated", "email": "ammar2222@gmail.com", "phone": "+62 123-4567-8910", "balance": 50000 }`
	Body Cookies Headers (21) Test Results 200 OK 112 ms
	{} JSON Preview Visualize
	`{ "success": true, "message": "User updated successfully", "payload": { "id": 1, "name": "Ammar Updated", "username": "alice", "email": "ammar2222@gmail.com", "phone": "+62 123-4567-8910", "balance": 50000 } }`

Postman get sudah bukan dari cache lagi	GET http://localhost:3000/user/user/ammar2222@gmail.com Docs Params Authorization Headers (9) Body Scripts Settings none form-data x-www-form-urlencoded raw binary GraphQL JSON { "id": 1, "name": "Ammar Updated", "email": "ammar2222@gmail.com", "phone": "+62 123-4567-8910", "balance": 50000 } Body Cookies Headers (21) Test Results 200 OK {} JSON Preview Visualize { "success": true, "message": "User retrieved successfully", "payload": { "id": 1, "name": "Ammar Updated", "username": "alice", "email": "ammar2222@gmail.com", "phone": "+62 123-4567-8910", "password": "$2b$10$9DtbGXv7rqf3IXw7LUCSU.3GdeBc6mnOSVqZFvajImIlWS/DW9vC.", "balance": 50000, "created_at": "2026-04-16T01:53:04.779Z" } }

4.

Langkah	Screenshot

transaction controller	`static async createTransaction(req, res, next) {`
	`try {`
	`const { user_id, item_id, quantity, description } = req.body;`
	`const transaction = await TransactionService.createTransaction({ user_id, item_id, quantity, description });`
	
	`await redis.xadd(`
	`'transaction-logs',`
	`'*',`
	`'userId', String(user_id),`
	`'itemId', String(item_id),`
	`'total', String(transaction.total_price)`
	`);`
	
	`res.status(201).json({`
	`success: true,`
	`message: 'Transaction created successfully',`
	`payload: transaction,`
	`});`
	`} catch (error) {`
	`next(error);`
	`}`
	`}`

Postman POST	POST http://localhost:3000/transaction/create
	Docs Params Authorization Headers (9) Body Scripts Settings
	none form-data x-www-form-urlencoded raw binary GraphQL JSON
	{
	"user_id": "1",
	"item_id": "2",
	"quantity": 1
	}
	Body Cookies Headers (21) Test Results 201 Created
	{} JSON Preview Visualize
	{
	"success": true,
	"message": "Transaction created successfully",
	"payload": {
	"id": 10,
	"user_id": 1,
	"item_id": 2,
	"quantity": 1,
	"total": 50000,
	"status": "pending",
	"description": null,
	"created_at": "2026-04-16T04:02:04.465Z"
	}
	}

Console	[Redis Stream Log] with Message ID 1776312124475-0 POST /transaction/create 201 - 54ms

5. Redis memiliki latensi yang lebih rendah karena data yang ‘sering’ dikunjungi disimpan pada sebuah cache di memory yang mana menyebabkan akses pada data lebih cepat dibandingkan dengan menggunakan query langsung ke postgreSQL yang mana memerlukan akses secara ‘eksterna’